

Experimental Evaluation of Implementation-Level Refinements for Trajectory Tracking in a SCARA Manipulator

Santiago Estrada  and Manuel-Ortiz 

Abstract—This paper presents the design and experimental evaluation of a custom 3-DOF RRP SCARA manipulator as a platform for studying trajectory execution under practical implementation constraints. A discrete-time cascade control architecture is implemented, combining joint-space position–velocity control with encoder-based feedback and lightweight state estimation. Rather than introducing new control laws, the work focuses on systematically assessing the impact of implementation-level refinements, including mechanical calibration, trajectory interpolation, timing determinism, and velocity estimation, through a comparison with a baseline configuration. Experimental results, obtained from both qualitative trace analysis and quantitative tracking metrics, show clear improvements in geometric fidelity, motion smoothness, and contact consistency. For geometrically complex trajectories, the final system achieves tracking errors on the order of a few tenths of a millimeter. These results highlight the importance of integrated mechanical, computational, and control design in achieving reliable trajectory execution in low-cost robotic platforms.

Index Terms—SCARA manipulator, cascade control, discrete PID control, Kalman filtering, trajectory tracking

I. INTRODUCTION

CUSTOM-built robotic platforms play an important role in bridging theoretical control design and real-world implementation, particularly in academic and low-cost settings where system integration constraints significantly affect performance. In such platforms, trajectory execution quality is often determined not only by the control law, but also by practical factors including mechanical accuracy, sensing quality, timing consistency, and software architecture. Despite their importance, the combined impact of these implementation-level aspects is rarely evaluated in a systematic experimental manner.

SCARA manipulators are widely used for planar trajectory tracking tasks due to their structural simplicity and computationally efficient kinematics. While model-based control strategies can achieve high accuracy under well-identified conditions [1], their performance may degrade in low-cost systems affected by parametric uncertainty, friction, and actuator limitations. For this reason, cascaded position–velocity control architectures based on discrete-time PID controllers remain a common practical solution [2]. However, their performance is strongly influenced by implementation details such as derivative discretization, velocity estimation, and sampling consistency, particularly when feedback is derived from encoder measurements [3], [4].

This work addresses this gap by experimentally evaluating how implementation-level refinements affect trajectory execution in a custom SCARA manipulator. A complete system is developed, including mechanical design, embedded control, and trajectory generation, and a discrete-time cascade control architecture is implemented with noise-aware derivative action and encoder-based state estimation. Rather than proposing a new control strategy, the focus is placed on comparing a baseline configuration with a progressively refined system incorporating mechanical calibration, improved interpolation, deterministic timing, and filtered velocity feedback.

The main contribution of this paper is a structured experimental assessment of how these combined refinements influence trajectory tracking performance in a low-cost embedded robotic system. Results demonstrate consistent improvements in geometric accuracy, motion smoothness, and contact stability across both qualitative and quantitative evaluations, while also revealing residual limitations associated with the system design.

The remainder of this paper is organized as follows. Section II briefly summarizes the modeling, trajectory generation, and control framework. Section III describes the robot design and system architecture. Section IV presents the experimental results and analysis. Conclusions and future work are discussed in Section V.

II. THEORY AND METHODOLOGY

A. Kinematic Modeling

The manipulator kinematics follow the standard SCARA formulation, consisting of a planar two-revolute (2R) chain for horizontal motion and a prismatic joint for vertical displacement. Forward kinematics are derived using Denavit–Hartenberg (DH) conventions, yielding the end-effector pose through the composition of homogeneous transformations [5]. The planar position depends on the calibrated link lengths $r_1 = 0.16$ m and $r_2 = 0.143$ m, while the vertical coordinate is determined by the prismatic displacement with a constant offset.

Inverse kinematics for the planar subsystem are obtained in closed form. For a desired Cartesian position (x, y) ,

$$D = \frac{x^2 + y^2 - r_1^2 - r_2^2}{2r_1r_2}, \quad (1)$$

with $\theta_2 = \pm \arccos(D)$ corresponding to the two possible configurations. The angle θ_1 is computed from the corresponding geometric relations, and the prismatic displacement is obtained directly from the desired vertical coordinate.

Feasible configurations satisfy $|D| \leq 1$ and joint limits imposed by the mechanical design. A consistent elbow-down

Santiago Estrada and David Roza are with EIA University, Envigado, Colombia (e-mail: santiago.estrada6@eia.edu.co; manuel.ortiz@eia.edu.co).
Corresponding author: Santiago Estrada.

configuration is selected to ensure continuous trajectory execution.

B. Trajectory Generation and Mapping

Cartesian reference trajectories are defined in the planar workspace as geometric paths (e.g., circular and sprocket-like profiles), independently of the robot kinematics. The discrete paths are interpolated using cubic splines to obtain a continuous representation, yielding a dense sequence of Cartesian waypoints (x, y, z) .

The interpolated waypoints are mapped to joint space via inverse kinematics, producing a sequence of joint configurations consistent with the desired workspace geometry. To improve execution smoothness, redundant configurations are filtered based on a minimum angular variation criterion, and the remaining points are interpolated in joint space using cubic polynomials with zero boundary velocities.

The prismatic joint is treated separately and follows a piecewise-constant reference corresponding to contact and transition phases.

This pipeline maps workspace-defined trajectories into smooth joint-space references suitable for execution.

C. Control and State Estimation

The control architecture follows a joint-independent cascade structure, where an outer position loop generates velocity references for an inner velocity loop. No explicit dynamic model of the manipulator is used, emphasizing an implementation-oriented control design. The position loop operates at an update rate 10 times slower than the velocity loop.

1) *Discrete PID Velocity Control*: The inner loop is implemented as a discrete-time PID controller acting on joint velocity,

$$u[k] = K_p e[k] + K_i \sum_{i=0}^k e[i] + K_d \frac{e[k] - e[k-1]}{T_s} \quad (2)$$

where $e[k]$ is the velocity tracking error and T_s is the sampling period. The controller is implemented in incremental form for embedded execution [4]. To mitigate noise amplification, the derivative term is realized using an impulse-area invariant discretization [3], resulting in a filtered differentiator behavior that improves robustness under encoder-based feedback.

2) *State Estimation via Kalman Filtering*: Joint velocity is estimated using a single-state discrete-time Kalman filter based on a first-order motor model,

$$\hat{\omega}[k] = a \hat{\omega}[k-1] + b u[k-1] \quad (3)$$

where (a, b) are obtained from a discrete-time motor representation. The estimator combines model-based prediction with encoder-derived measurements through a prediction-correction scheme, with correction potentially applied at a lower rate than prediction to accommodate measurement limitations.

The estimated velocity is used as feedback for the inner loop, while the outer position loop provides velocity references. This integration reduces the effect of measurement noise without compromising control responsiveness.

III. ROBOT DESIGN AND SYSTEM ARCHITECTURE

A. Hardware

The system is a 3-DOF RRP SCARA manipulator comprising two coplanar revolute joints for planar motion and a vertically actuated prismatic joint. The mechanical structure is primarily fabricated from laser-cut acrylic, assembled in a modular configuration with selected additively manufactured components.

The workspace spans approximately 300 ± 20 mm in both horizontal and vertical directions. The planar links have nominal lengths $r_1 = 16$ cm and $r_2 = 12$ cm (later refined through calibration), with joint limits of $\pm 90^\circ$ and $\pm 120^\circ$ for the first and second revolute joints, respectively. Belt-driven transmissions are used to reduce backlash.

The revolute joints are actuated by DC motors with integrated quadrature encoders, while the prismatic joint is driven by a servo actuator. Low-level control is implemented on a microcontroller, and all electronics, including power regulation and motor driving, are integrated on a custom PCB.

B. Software and Control Architecture

The embedded software was implemented in C++ using a modular architecture that encapsulates actuation, sensing, control, and communication functionalities into reusable components. The embedded controller operates as a `micro-ROS` node, while ROS 2 runs on a host PC, with communication handled through the `micro-ROS` middleware.

Execution on the embedded system follows a main-loop structure combining communication handling with hardware-timed control tasks, ensuring deterministic timing for control and estimation tasks. The system receives joint-space references consisting of two revolute angles and a prismatic displacement, while joint measurements are returned for monitoring and evaluation.

The two revolute joints are controlled using a joint-independent cascade architecture, where position errors are mapped to velocity references and tracked by inner velocity controllers. Motion coordination arises from the concurrent execution of joint commands, without explicit Cartesian-level control or dynamic coupling. The prismatic joint is controlled independently through its internal servo loop.

State estimation is integrated within the velocity control layer. Each joint incorporates a Kalman-based velocity estimator driven by encoder measurements and actuation inputs, providing filtered feedback for control. The impact of this estimator is evaluated experimentally in Section IV.

IV. RESULTS AND ANALYSIS

A. Initial Setup

The baseline configuration employed the custom 3-DOF RRP SCARA manipulator described in Section III. Joint positions were obtained from the integrated encoders of the revolute joints, and the end-effector pose was estimated via forward kinematics, with no direct Cartesian sensing.

Trajectory generation and kinematic computation were performed on a host PC. Joint-space references were generated

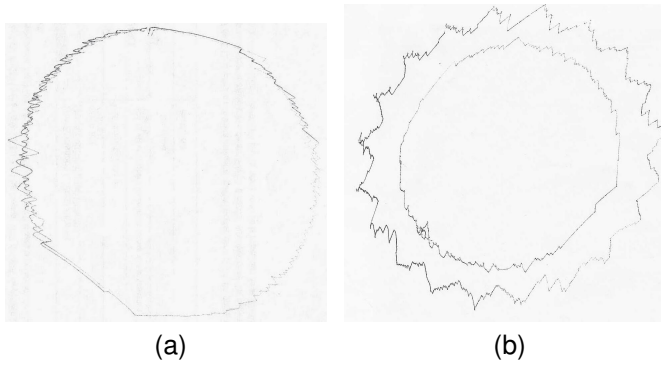


Fig. 1. Best traces obtained with the initial setup. (a) Circle. (b) Sprocket.

online and streamed to the embedded controller at 60 Hz, while joint measurements were returned for forward kinematics and real-time visualization. No data logging was performed in this configuration. On the embedded side, both position and velocity control loops were executed at 20 Hz, with motor actuation via PWM at 10 kHz. This setup defines the baseline for comparison with the improved configurations.

Control followed the joint-independent cascade structure described previously, with the prismatic joint controlled through its internal servo loop.

Trajectory execution consisted of repeatedly commanding predefined circular and sprocket paths, represented as joint-space waypoints obtained through cubic interpolation. No explicit timing constraints were enforced during execution. Performance evaluation was qualitative, based on visual inspection of the traces drawn by the end-effector on paper. Multiple trials were conducted, and Figs. 1a and 1b show the best-performing runs.

As shown in Fig. 1, the circular trajectory exhibits noticeable geometric distortion, including oval deformation and jagged segments with localized irregularities. Variations in line thickness indicate inconsistent contact during execution. The sprocket trajectory presents similar effects, with distortion at sharp features, loss of triangular definition, and localized overshoot near curvature discontinuities. These limitations motivate the refinements introduced in the following subsection.

B. Improvements to System Design and Control

This subsection summarizes the modifications introduced relative to the baseline configuration.

Mechanical refinements addressed geometric distortions and contact inconsistencies. The Denavit–Hartenberg parameters were calibrated through direct measurement of link lengths and joint offsets, accounting for discrepancies between the nominal design and the assembled structure. In addition, the prismatic joint was modified to include a passive spring mechanism that maintains continuous contact between the end-effector and the drawing surface, improving trace continuity without altering the kinematic structure.

Modifications to the software pipeline focused on trajectory quality and timing determinism. Trajectory generation and inverse kinematics were moved offline, and Cartesian paths

were interpolated using cubic splines. Joint-space interpolation was applied to smooth the resulting references. Setpoints were streamed deterministically to the embedded controller at 20 Hz using ROS 2 timers. On the embedded side, PWM frequency was increased to 20 kHz, and control loops were executed using hardware timers, decoupling them from communication callbacks.

Control refinements were introduced in the velocity and estimation layers of the cascade architecture. The velocity controller employed a discrete PID structure with impulse-area invariant discretization of the derivative term. Controller gains were retuned based on updated motor characterization, and output saturation was included to limit the PWM command and prevent integral windup.

Velocity feedback was improved through the inclusion of a single-state Kalman filter for each joint. The estimator is based on a first-order motor model identified from step-response data and uses the applied PWM signal as input. Prediction and control were executed at 5 ms intervals, while encoder-based correction was applied at 25 ms. The outer position loop operated at 50 ms, maintaining a 10:1 ratio between velocity and position updates. As in the baseline configuration, joints were controlled independently, with coordination achieved through position-level speed scaling, and the prismatic joint remained outside the cascade structure.

C. Final Configuration and Performance

The effectiveness of the implemented hardware, software, and control modifications was evaluated using circular and sprocket trajectories consistent with those of the baseline configuration. Although the references were regenerated to account for updated interpolation and communication rates, the execution procedure and workspace placement were preserved to ensure comparability. Performance was assessed through both qualitative trace analysis and quantitative tracking metrics.

Fig. 2 shows the traces obtained with the improved configuration. The circular trajectory exhibits a clear reduction in oval distortion and jagged segments, with smoother curvature and more uniform line thickness. Localized loss of contact observed in the baseline is no longer present, although small residual irregularities remain.

The sprocket trajectory preserves all geometric features, including clearly defined spikes and improved triangular structure. Deviations are primarily localized near sharp corners, while the inner circular region closely matches the corresponding standalone trajectory. Overall, the traces indicate improved geometric fidelity, smoothness, and contact consistency compared to the baseline.

Quantitative performance was evaluated using the sprocket trajectory due to its geometric complexity. The executed Cartesian trajectory, reconstructed from encoder measurements and calibrated kinematics, was compared with the reference path using nearest-neighbor matching in the (x, y) workspace. After removing initial and final transients, the root-mean-square error (RMSE) was 0.36 mm, with directional RMSE values of 0.22 mm in x and 0.28 mm in y . The mean tracking error was 0.27 mm, and the maximum deviation was 1.77 mm.

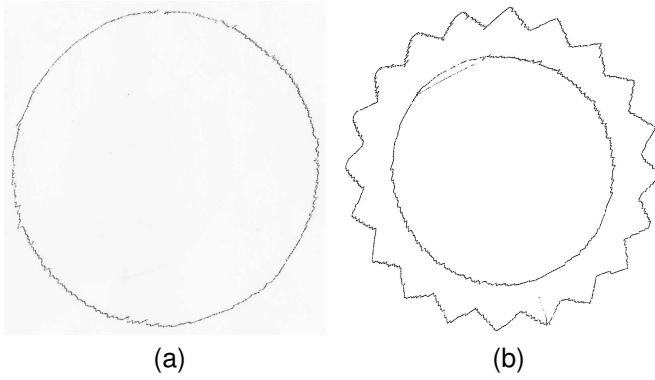


Fig. 2. Final traces obtained with the improved configuration. (a) Circle. (b) Sprocket.

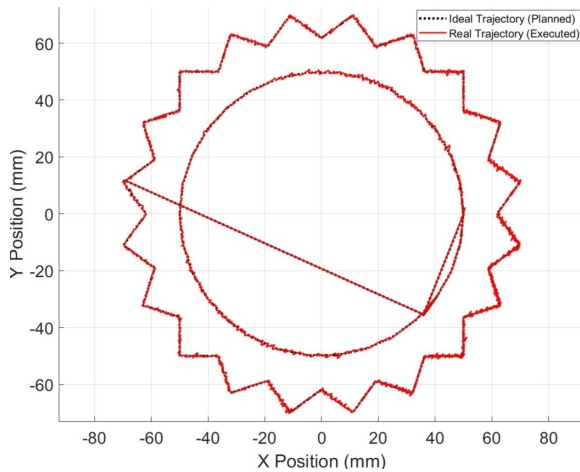


Fig. 3. Overlay of desired (dashed black) and executed (solid red) sprocket trajectories. Trajectories are translated to the origin for visualization; the sprocket geometry admits an enclosing circle of radius 70.5 mm.

Fig. 3 overlays the desired and executed trajectories. The executed path closely follows the reference, with small, distributed deviations and no large localized errors. The observed spatial deviations are consistent with the reported error metrics. For visualization, trajectories are translated to the origin, and the sprocket geometry is enclosed within a circle of radius 70.5 mm.

V. CONCLUSIONS AND FUTURE WORK

This paper presented the design and experimental evaluation of a custom 3-DOF RRP SCARA manipulator for trajectory execution under practical implementation constraints. The work focused on assessing how system-level refinements in mechanical design, trajectory generation, timing, and control affect tracking performance in an embedded robotic platform.

Experimental results showed that the baseline configuration exhibits noticeable geometric distortion and execution inconsistencies. The refined system achieved improved trajectory reproduction, with increased geometric fidelity, smoother motion, and more consistent end-effector contact, as observed in both qualitative traces and quantitative metrics.

For a geometrically complex trajectory, the final configuration achieved tracking errors on the order of a few tenths of a

millimeter, with deviations remaining bounded and distributed along the path. These results demonstrate that careful integration of mechanical calibration, trajectory processing, and control implementation can significantly improve performance in low-cost manipulators, without requiring model-based control.

Remaining limitations are primarily associated with mechanical compliance and the use of joint-space control without direct Cartesian feedback.

Future work will consider the integration of model-based compensation, inter-joint coupling, and direct Cartesian sensing, as well as evaluation under a broader set of trajectories and operating conditions.

ACKNOWLEDGMENTS

The authors would like to thank Alejandro Marín, Herman Franco, Nicolás Moreno, and Jose David Gómez for their invaluable assistance in the design and construction of the SCARA robot used in this study. We also thank EIA University for providing materials and laboratory equipment, and Jose Valentín Restrepo for his help in verifying the integrity of the custom-made PCBs.

REFERENCES

- [1] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot modeling and control*. Wiley New York, 2020, vol. 2.
- [2] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [3] M. Schmidt, "Derivative action in discrete pid controllers," 2007, pp. 101–105.
- [4] F. Haugen, "Discretization of simulator, filter, and pid controller," Telemark University College, 2010, available online.
- [5] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2010.